

Dynamic network-aware soft failure localization using machine learning in optical networks

VIGNESH KARUNAKARAN,^{1,2,*} RONALD ROMERO REYES,² BEHNAM SHARIATI,³ JOHANNES KARL FISCHER,³ ACHIM AUTENRIETH,¹ AND THOMAS BAUSCHERT²

¹Adtran Networks SE, Munich, Germany

²Chair of Communication Networks, TU Chemnitz, Chemnitz, Germany

³Fraunhofer HHI, Berlin, Germany

*vignesh.karunakaran@adtran.com

Received 8 April 2025; revised 8 October 2025; accepted 9 October 2025; published 24 October 2025

With the dynamic nature of optical service provisioning and network topology reconfigurations, failure identification and management become complex, as the machine learning (ML) model is trained for a specific topology with pre-defined performance metrics. This paper proposes a hybrid ML framework for continuous monitoring and soft failure (SF) localization in a partially disaggregated optical network. The framework combines a distributed unsupervised machine learning approach for per-device monitoring and an inductive graph neural network (GNN) for SF localization. This allows the system to generalize across dynamic network conditions, including optical service reconfigurations and node additions or deletions. To support real-time data collection and provide data plane visibility in the management plane, this work proposes gNMI/gRPC-based telemetry streaming using a unified ONF-TAPI YANG data model, enabling vendor-neutral communication across multi-domain networks. The proposed telemetry streaming outperforms the existing solution by reducing traffic load by a factor of 78.4%, and the inductive GNN-based failure localization maintains an accuracy of 97.4% despite dynamic network reconfigurations. © 2025 Optica Publishing Group. All rights, including for text and data mining (TDM), Artificial Intelligence (AI) training, and similar technologies, are reserved.

<https://doi.org/10.1364/JOCN.564177>

1. INTRODUCTION

With the significant increase in network traffic, network operators rely on the optical transport network infrastructure to support 5G and emerging services. Optical device vendors made commendable advancements in access, metro, and core network components [1]. These advancements include innovations in pluggable coherent optics, multi-band over spatial division multiplexing systems, digital signal processing, and increased data transmission rates (400G/800G). However, as networks integrate these advanced features, operators increasingly depend on device vendors for the control and management of the underlying infrastructure, leading to challenges when integrating devices from multiple vendors. To address this issue, network operators and vendors have collaborated to define generic open-source yet another next generation (YANG) data models for device control. However, with rapid advancements in feature sets, mapping and abstracting all features and device configurations into a generic data model remains challenging. As highlighted in [2], limitations exist in using the open-source OpenROADM YANG [3] to fully extract the capabilities of the network element (NE). This challenge led to the concept of a partially disaggregated optical transport network, where each optical line system (OLS)

domain is controlled by a well-informed vendor-specific OLS controller to utilize the full potential of network elements. The OLS domain controllers are controlled by a hierarchical software-defined networking (SDN) controller with a standardized YANG data model, such as ONF-TAPI [4]. This approach maintains the separation of the data, control, and management planes with challenges in monitoring and managing the network services across multiple domains from the management plane.

In a partially disaggregated network, RESTCONF is used for communication in the northbound, which is the interface between the management plane (MP) and the control plane (CP), while NETCONF is employed in the southbound, which is the interface between the CP and the data plane (DP) during network operation [5]. While device configuration for service provisioning is simplified, monitoring DP components is essential for effective network management and automation. This adds complexity in retrieving performance monitoring (PM) data from a partially disaggregated network using appropriate protocols and standard data models.

Existing approaches use SNMP or NETCONF protocol for both device configuration and PM data retrieval. However, this is less efficient due to the text-based serialization format

and increased payload size. Alternatively, the gRPC protocol, which is widely used to establish communication between microservices, is also used for monitoring due to its streaming functionality and effective serialization [6,7]. In this work, we investigate the end-to-end use of the gNMI/gRPC protocol in compliance with TAPI v2.5 for telemetry streaming and evaluate its efficiency.

This paper investigates dynamic network-aware failure localization with enhanced telemetry streaming in optical transport networks. Given the diversity of services carried by these networks, each with different service level agreements (SLAs), continuous monitoring of PM data is necessary to maintain the SLAs. To achieve this, a hybrid ML framework is proposed, combining unsupervised ML for anomaly detection and graph neural network for soft failure classification. Since different optical channel configurations can be provisioned to carry the services, and the device key performance indicators (KPIs) may vary due to differences in channel configurations, we identify unsupervised ML as the most suitable strategy for failure detection, simplifying feature selection and data training [8]. Within this context, each optical device is assigned a dedicated ML instance that performs anomaly detection in a distributed manner. These identified anomalies are then processed using a GNN-based model for SF classification and localization. The main contributions of this work are summarized as follows:

- An end-to-end gNMI/gRPC-based telemetry streaming using a standard data model to improve the data plane visibility in a partially disaggregated optical network.
- A novel hybrid ML framework to identify and localize soft failures. The work illustrates the effectiveness of an unsupervised ML algorithm with reduced training time and supports dynamic optical service configurations. The proposed inductive GNN model for SF classification supports changes in network topology and proactive failure localization.

2. STATE OF THE ART

Network automation in optical networks is rapidly evolving, with various use cases such as ML-based resource allocation [9], fault prediction and automated recovery, and energy-efficient optimization [10]. The challenges remain in partially disaggregated networks, particularly regarding performance data streaming and utilizing telemetry data for network management.

The work in [11] experimentally validates the control plane for a multi-domain network, where network elements are compatible with the OpenROADM YANG data model to abstract the device capabilities. The approach evaluates multiple domains with the same YANG model and demonstrates end-to-end network media channel provisioning using the ONF-TAPI specification. In [12], the authors discuss telemetry streaming from open ROADMs in a fully disaggregated network. It focuses on extending NETCONF capabilities in ONOS-based SDN controllers to support sub-second data streaming. Both [11,12] have gaps in addressing data streaming to the management plane, which is crucial for operators managing cross-domain networks. The partially disaggregated

multi-domain network considered in [13] uses transceivers controlled by OpenConfig, with two WDM domains managed by respective OLS controllers. The work adds the novelty of integrating an SDM OLS domain in the hierarchical multi-domain SDN architecture with a limited focus on streaming and ML-based solutions. Building on [13], the study presented in [6] extends optical monitoring and streaming telemetry in a partially disaggregated optical transport network. It investigates device-based streaming with gNMI and controller-based streaming using TAPI RESTCONF notifications. The gaps in [6,13] include telemetry streaming with suitable protocols and improving data plane visibility.

The study in [14] presents an ETSI-based telemetry framework for a partially disaggregated optical network and demonstrates distinct ML applications for traffic forecasting, traffic pattern recognition, and anomaly detection of PON devices. Telemetry data from the optical transport network are retrieved using NETCONF and stored in the data lake for processing. The paper provides hints on improving telemetry streaming in partially disaggregated networks and a real-time analysis framework for network management. The work in [15] uses GNNs to localize the fault based on the alarm data from the nodes. It mainly addresses the problem of identifying the cause of alarms from nodes using ML, with prior knowledge of the relationships between the alarms. The authors of [16] proposed a federated ML model for a partially disaggregated network for soft failure localization, where the locations traversed by the lightpath correspond to failure classes. The work in [17] proposes a multi-failure localization in high-degree ROADMs using rules-informed NNs similar to knowledge graph-based NNs as in [15]. The contributions in [15–17] are limited to fixed network topology and remain challenging for dynamic network scenarios, as mentioned in the respective works. The authors of [18] proposed a distributed database (DB) strategy where each device is assigned a local DB instance to collect telemetry via gRPC/NETCONF. Data are pushed to a central DB for anomaly detection and QoT prediction using long short-term memory (LSTM) and an ANN. While the work provides a unified monitoring solution, it is limited to a known topology and lacks root cause analysis. Reference [19] explores the integration of SDN, telemetry, and AI, emphasizing the potential of gNMI/gRPC for efficient telemetry streaming, and challenges related to the lack of standardization in multi-vendor deployments, which are crucial for achieving interoperability and enabling robust AI/ML-driven solutions that are addressed in this work.

A survey of different state-of-the-art solutions is summarized in Table 1. Our proposed work addresses the difficulties in telemetry streaming within a partially disaggregated network, where PM data are critical for monitoring cross-domain services. With a unified data model, the network operator can monitor the PM data across the provisioned service path, streaming from each optical device. This will simplify failure identification in the network, as it increases PM data visibility for each optical device. The dynamic aspect of optical service provisioning refers to the capability to re-provision services when triggered by practical requirements such as spectrum defragmentation, scheduled optimization, and channel capacity adjustment [9,10,20]. This increases the complexity of

Table 1. Survey on Existing Works Related to Monitoring and Control of Optical Transport Network

Ref. No.	Multi-Domain		Protocol			YANG Model				Telemetry Streaming	Failure Identification/Localization
	Partially Disagg.	Fully Disagg.	NET-CONF	REST-CONF	gRPC	Open-Config	Open-ROADM	Native	ONF-TAPI		
[11]		✓	✓				✓			✓	
[12]			✓							✓	
[13]			✓						✓	✓	
[6]			✓			✓			✓	✓	
[14]			✓			✓		✓		✓	✓
[15]				✓						✓	✓
[16]										✓	✓
[17]										✓	✓
[18]				✓	✓	✓				✓	✓
[19]				✓	✓	✓				✓	✓
This paper	✓			✓	✓	✓		✓	✓	✓	✓

ML-based failure localization as it poses a challenge in training data for new service configuration or topology update.

3. BACKGROUND

A. Challenges in Monitoring a Partially Disaggregated Network

As network use cases continue to evolve, model-driven approaches based on YANG data models are adopted today to perform device configurations. However, operators face difficulties in managing devices from different vendors as each has its own proprietary YANG definitions. To address this, operators, service providers, and vendors are collaborating together to define open-source YANG models such as OpenConfig [21] and OpenROADM [3]. Despite these efforts, device manufacturers are finding it difficult to map each device configuration and operational parameters to open-source YANG definitions [2]. This issue can be mitigated by adopting a partially disaggregated network architecture, where each OLS domain is controlled by an independent OLS controller, which in turn is controlled by a hierarchical SDN controller.

Monitoring and controlling an OLS from the MP is of particular interest to operators. For this, the transport API (TAPI) YANG data model can be used to abstract interface, device, service, and topology details among multiple vendor domains. Within an OLS domain, the control and data plane components are typically provided by the same vendor, where the use of native YANG definitions is more appropriate to fully exploit the capabilities. The interface between the MP and the CP of a given OLS domain is intended for high-level network operations. This interface can be realized through the use of open-source YANG definitions such as ONF-TAPI. This has been justified by the fact that the OLS controllers and the hierarchical SDN controller are often from different infrastructure providers. One drawback of this approach is that the multi-domain controller has limited visibility of both the network state and the performance monitoring data of the DP, resulting in reduced transparency. In this paper, we overcome this limitation by the definition of a unified data model that efficiently disseminates PM data from the DP to the MP. The YANG data model and its application in the partially disaggregated network can be better understood from Fig. 1 and Table 2.

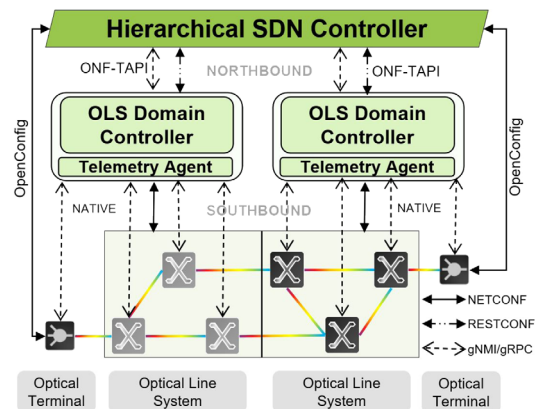


Fig. 1. Proposed architecture—partially disaggregated optical transport network.

Table 2. YANG Models and Their Usage in SDN Hierarchy

S. No	YANG Model	Protocol	Usage Between
1.	OpenConfig	NETCONF	DP and CP, DP and MP
2.	OpenROADM	NETCONF	DP and CP
3.	Native/Vendor	NETCONF	DP and CP
4.	ONF-TAPI	RESTCONF	CP and MP

Table 3. Protocols and Their Usage in SDN Hierarchy

S. No	Protocol	Usage Between	Configuration	Telemetry
1.	SNMP	DP and CP	Yes	Yes
2.	NETCONF	DP and CP, DP and MP	Yes	Yes
3.	RESTCONF	CP and MP	Yes	Yes
4.	gRPC	DP and CP, CP and MP	No	Yes

Concerning flexible configuration management, transaction consistency, and data model extension, NETCONF has been preferred over SNMP in SDN architectures. However, network monitoring and automation require seamless telemetry data exchange, which has been shown to be inefficient with SNMP, NETCONF, and RESTCONF [22]. Although NETCONF and RESTCONF are effective for configuration operation, they are traffic-intensive and latency-sensitive [23]. In contrast, a protocol like gRPC has better data encoding formats using binary serialization and offers a more effective subscription model compared to NETCONF and RESTCONF. An overview of the protocols used in a partially disaggregated optical transport network and their application is shown in Table 3.

B. Challenges in Soft Failure Localization

With the need for dynamic optical service provisioning, network reconfiguration, and optimization decisions to ensure efficient resource usage, real-time failure management is essential to maintain the SLA. Soft failure degrades the quality of transmission (QoT) without triggering explicit alarms, unlike hard failures, which are clearly notified. Although device-level thresholds can raise warnings, these are typically

vendor-defined global settings and cannot capture gradual degradations. Such degradations can instead be identified by continuously monitoring the KPIs of all optical devices. Existing work [18,24] focuses on optical power monitoring across the path for SF detection, while we adapt a similar approach with additional KPIs for monitoring, as shown in Table 4. However, monitoring multiple parameters introduces complexity with varying distribution and limited availability of appropriate training data due to the network dynamicity. To overcome these limitations, we propose the adoption of an unsupervised ML model for the identification of SF, which requires fewer training data and does not rely on a labeled dataset.

A high-degree ROADM constitutes (i) intra-connected components, including power splitter modules (PSMs), wavelength-selective switches (WSSs), and pre- and booster amplifiers, and (ii) inter-connected with ROADMs across multiple fiber spans. A degradation in the QoT of the device can cause sequential components to deviate from the ideal performance, resulting in anomalies across multiple nodes. Therefore, identifying the relationship between affected nodes is critical to localize and determine the type of SF. Existing works [15,17] propose ML models based on GNN to capture node relationships for fault localization. However, these approaches are often limited to reacting after alarms are triggered or assume a fixed network topology. In contrast, dynamic ML models could provide a solution for learning as the network evolves. However, continuously training an ML model with updates in topology and service and real-time SF localization is complex. We propose a hybrid ML framework that combines unsupervised and supervised ML models for failure identification and localization addressing the aforementioned gaps.

4. PROPOSED FRAMEWORK: TELEMETRY STREAMING AND SOFT FAILURE LOCALIZATION

A. E2E Streaming Telemetry Solution

We propose end-to-end gNMI/gRPC-based telemetry streaming, replacing NETCONF, RESTCONF, and SNMP for performance data exchange. gNMI [25] is a protocol specification developed by the OpenConfig working group for network

Table 4. Telemetry Streaming of OT and OLS Components

S. No	Component Type	Component	Parameter
1.	Optical terminal	Network port	Signal to noise ratio, Q-factor, optical power TX, optical power RX, BER, carrier frequency offset, differential group delay, chromatic disp. compensation, laser bias current, laser temperature
		Client port	Optical power RX, BER
2.	Optical line system	Amplifier	Client port ip. pwr, client port op. pwr, network port ip. pwr, network port op. pwr, ROADM port ip. pwr, ROADM port op. pwr, booster amp total ip. pwr, booster amp total op. pwr, booster amp target gain, booster amp actual gain, booster amp target-gain-tilt, booster amp actual-gain-tilt, pre-amp mid-stage ip. pwr, pre-amp mid-stage op. pwr, operation status
		WSS	Output power, operation status
		Power splitter module	Operation status equipment failure

management. It defines a standard API for interaction with devices using gRPC and protocol buffers for efficient data serialization and transport.

1. Southbound: Between DP and CP Components

NETCONF, an existing approach for telemetry data exchange, uses SSH as its transport layer for communication, whereas gNMI/gRPC is built on HTTP/2 with native streaming support using a subscription model. We propose a gNMI/gRPC approach in which the control plane telemetry agent (gRPC client), as shown in Fig. 1, subscribes to telemetry data from the devices (gRPC server). The client uses the vendor's XPath definitions for subscription request and receives data periodically. The communication sequence, payload, and overhead of NETCONF and gNMI/gRPC-based data retrieval are compared in Figs. 2(a) and 2(b) for the same set of PM data from an optical terminal as shown in Table 4. It is observed that the payload size of the request and response in gNMI/gRPC is smaller than in NETCONF by a factor of $3.5\times$ and $1.7\times$. Additionally, the overhead size in gNMI/gRPC is smaller than in NETCONF, with reductions of about $2\times$ for requests and $1.5\times$ for responses. A key difference between gNMI/gRPC and NETCONF is that gRPC establishes a subscription session and continuously receives PM data at regular sample intervals, thereby conserving uplink traffic on the client.

2. Northbound: Between CP and MP Components

The OT and OLS components use native vendor-based definitions for streaming PM data, which limits the DP visibility to MP applications. We implemented the agent mentioned in Section 4.A.1 also facilitates the exchange of performance data between different domain controllers in the CP and the hierarchical SDN controller in the MP using a unified data model. This would improve DP visibility and enable various ML and intelligence applications in the MP to automate and manage the network efficiently. We evaluated the ONF-TAPI v2.5.1 model [4], a standard API framework designed to provision and manage the service across the network domains for telemetry streaming between the CP and MP. The existing release of ONF-TAPI defines a RESTCONF-based polling

mechanism to retrieve telemetry data and a notification mechanism for alarm data. However, the telemetry data exchange using RESTCONF/JSON is verbose and less efficient in terms of latency, overhead, and resource usage.

From a southbound perspective, data collection is limited to a specific domain. However, in the northbound, it is a multi-domain scenario, which leads to the following issues:

- Volume of data: Performance data sampled at regular intervals result in a massive data volume. gRPC uses binary data formats and effective serialization techniques, making the payload significantly smaller [23].
- Effective streaming: Given the vast performance data measurements, it is necessary to (i) eliminate data duplication, (ii) avoid invariant fields, and (iii) transmit only updated data.
- Data sources: Collecting data from multiple domains is a challenge for network operators, which require a unified data model for data retrieval.

In addition to these issues, TAPI v2.5.1 discusses several other concerns in Section 6.A of [4]. We implemented REST and gNMI/gRPC according to the TAPI data model and compared data retrieval with both approaches. The respective communication flow to retrieve data from an optical terminal can be seen in Figs. 2(c) and 2(d). The polling mechanism used in RESTCONF requires a new connection to retrieve PM data at regular intervals. The payload size of request and response in the gNMI/gRPC approach is smaller than that of RESTCONF by a factor of $1.4\times$ and $2.3\times$. Additionally, the overhead in gNMI/gRPC is significantly smaller and is half the size of the overhead in RESTCONF.

The telemetry agent, shown in Fig. 1, is implemented to subscribe to PM data from devices within a vendor domain and also functions as a *tapi-gnmi-streaming* server [4]. From the MP, the TAPI gNMI/gRPC client subscribes to the device's PM data and stores it in a time-series DB managed by the hierarchical SDN controller. The gNMI specification provides the API definition for subscription operations, and the TAPI model defines a unified request and response format to ensure compliance among different domains in a partially disaggregated network. For devices lacking native telemetry streaming capabilities in a multi-vendor network, flexible telemetry

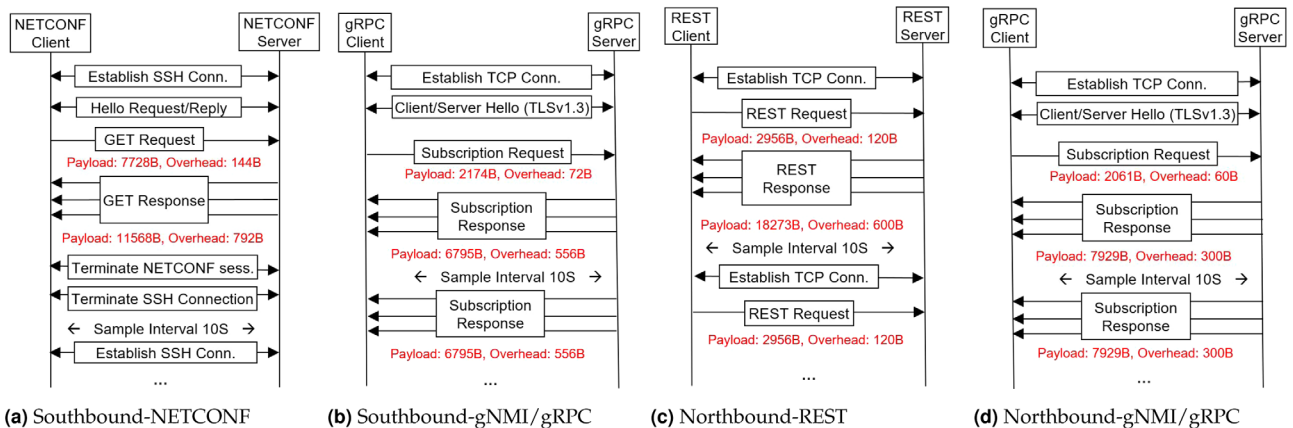


Fig. 2. Telemetry data retrieval in southbound (between the DP and the CP) and northbound (between the CP and the MP).

agents enable DP visibility toward MP via a TAPI-based plugin in the northbound interface [19,26]. A configurable sample interval of 10 s is used to receive subscribed data in both NB and SB interfaces.

B. Failure Identification Using Unsupervised ML

We propose a distributed ML framework to process multifaceted PM data from various optical data plane devices. The objective is to identify soft failures that cause deviation in performance metrics and are detected as anomalies by the ML model. As discussed in the previous section, the PM data acquired via telemetry streaming are periodically collected and stored in the time-series database. The data are subsequently made available for further processing from data-driven network applications, such as the anomaly detection framework proposed in this work. Nonetheless, deriving insights from PM dataset is challenging for the following reasons:

- The dataset may contain a mix of both configuration (static) and operational (dynamic) data.
- The PM data may consist of observations from multiple variables that may possess different statistical properties.
- Data observations from different variables may differ in both size (i.e., data volume) and sampling frequency.

The proposed ML framework tackles these challenges with the aid of unsupervised learning, which offers various anomaly detection techniques, such as clustering and distance-based methods. Previous work has applied density-based spatial clustering of applications with noise (DBSCAN) for anomaly detection [27]; however, tuning its parameters across heterogeneous KPIs is challenging. In this work, we adopt ordering points to identify the clustering structure (OPTICS), which overcomes this limitation by leveraging the reachability plot to capture clustering structure across varying data distributions.

1. OPTICS-Based Anomaly Detection

For each optical device, the corresponding telemetry streaming data (retrieved via gNMI/gRPC) are stored in a time-series DB for further processing with the OPTICS algorithm. For that, each PM reported by the OT and OLS devices from their client ports, line ports, and logical interfaces is considered as an individual feature, thereby defining a multi-dimensional dataset. With this dataset, OPTICS is applied to define data clusters based on the density of the data. The advantages of OPTICS over other clustering methods are that it works well with clusters with varying density, it enables automatic identification of clusters, and it unveils the relationships between different clusters. Besides, it allows the identification of noisy observations.

The input parameters of the OPTICS algorithm are the minimum radius ϵ and the minimum number of points ζ required to form a neighborhood or a cluster. Figure 3 shows an example cluster in which the points represent the sample observations. The algorithm calculates the following parameters for each data point p :

- **Neighborhood of p :** It is the set of points that fall within the ϵ radius as measured from the data point p .

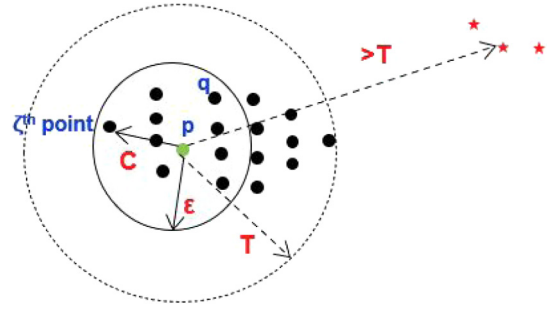


Fig. 3. OPTICS clustering—algorithm.

- **Core distance C of p :** Distance between p and the point that belongs to the minimum number of points ζ that is closest to p .
- **Reachability distance:** Distance between p and a neighboring point q . It is the maximum of the core distance C and the distance between p and q .

To start, the OPTICS algorithm initializes for each data point, its cluster number, core, and reachability distances as undefined values. Subsequently, the algorithm attempts to calculate a neighborhood for each data point based on the values of ϵ and ζ . In general, this procedure results in clustered and unclustered data, in which the latter corresponds to data points for which it is not possible to identify suitable clusters, and thus, these observations are identified as anomalies. This clustering procedure is explained in detail in the following.

Let X be an $m \times n$ matrix containing the telemetry data streamed by an optical device. The matrix is defined as

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \dots & \dots & \dots & \dots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix}_{m \times n}. \quad (1)$$

Each matrix row represents a sample associated with a given timestamp, with the sample consisting of n features. Each column corresponds to a feature defined by a PM metric. Thus, the matrix element x_{ij} represents the value of the j th PM metric observed at timestamp i . Let X_j be the column vector that represents the samples that correspond to the j th PM metric. Thus, the data in Eq. (1) can be expressed as $X = [X_1, X_2, X_3, \dots, X_n]$. From this definition, each data x_{ij} in X can be normalized as follows:

$$x'_{ij} = \frac{x_{ij} - \min(X_j)}{\max(X_j) - \min(X_j)}. \quad (2)$$

The resulting normalized data are therefore

$$X' = \begin{bmatrix} x'_{11} & x'_{12} & \dots & x'_{1n} \\ x'_{21} & x'_{22} & \dots & x'_{2n} \\ \dots & \dots & \dots & \dots \\ x'_{m1} & x'_{m2} & \dots & x'_{mn} \end{bmatrix}_{m \times n} = \begin{bmatrix} p_1 \\ p_2 \\ \dots \\ p_m \end{bmatrix}_{m \times 1}. \quad (3)$$

In general, the PM metrics may significantly differ among themselves in the scale or order of magnitude of the numerical values they may take on. Since OPTICS relies on the concept of distance, PM metrics with larger values (or orders of magnitude) dominate, and thus, skew the distance calculation.

This bias effect is avoided by applying the min–max scaler technique in [28], which normalizes the data by re-scaling all PM metrics between 0 and 1.

Each sample, i.e., row, in X' represents an n -dimensional observation that consists of n PM values or features. For the sake of simplicity, the i th sample can be represented by the n -dimensional point: $p_i = [x'_{i1}, x'_{i2}, \dots, x'_{in}]$. The normalized data are thus expressed as

$$X' = [p_1, p_2, \dots, p_m]^T. \quad (4)$$

With OPTICS, the radius ε of the neighborhood around a data point $p_i \in X'$ can be set as a value large enough as to avoid including anomalous points within the cluster. The neighborhood $N_\varepsilon(p_i)$ around p_i is the set of data points $q_j \in X'$ defined as

$$N_\varepsilon(p_i) = \{q_j \in X' | d(p_i, q_j) \leq \varepsilon\}, \quad (5)$$

where $d(p_i, q_j)$ is the distance between p_i and q_j . The number of points within the neighborhood should at least be equal to the minimum number of points required to form a cluster, i.e., $|N_\varepsilon(p_i)| \geq \zeta$. Moreover, the core distance $C_\varepsilon(p_i)$ of p_i is

$$C_\varepsilon(p_i) = \min_{q_j \in N_\varepsilon(p_i)} d(p_i, q_j). \quad (6)$$

The reachability distance $R(p_i, q_j)$ between p_i and any point $q_j \in N_\varepsilon(p_i)$ is calculated as

$$R(p_i, q_j) = \max(d(p_i, q_j), C_\varepsilon(p_i)), \quad \forall q_j \in N_\varepsilon(p_i). \quad (7)$$

Based on the above-mentioned definitions, the OPTICS algorithm applies Eqs. (5)–(7) to calculate—for the normalized data X' of each optical device—the distances $C_\varepsilon(p_i)$ and $R(p_i, q_j)$ of each n -dimensional data point $p_i \in X'$. Each optical device has a dedicated instance of OPTICS forming a cluster with the recent PM data X . The reachability distances across all the points $p_i \in X'$ are considered, and the maximum value is used as threshold reachability value T which is defined as

$$T = \max(R(p_i, q_j)) | \forall p_i, q_j \in X'. \quad (8)$$

The real-time data retrieved using gNMI/gRPC-based streaming are processed and stored in a time-series DB for further data analytics. These live data points are periodically retrieved and tested for anomalies. Let Y be the recent PM data representing all PM data values $Y = [y_{11}, y_{12}, y_{13}, \dots, y_{1n}]$ and Y' be the normalized data using the same scaler as shown in Eq. (2). The normalized data are expressed as

$$Y' = [y'_{11}, y'_{12}, y'_{13}, \dots, y'_{1n}] = [y']. \quad (9)$$

The reachability distance of the new data point $R(y', p_i)$ is the minimum distance between y' and the sample data points $p_i \in X'$ in the dataset. The sample points are retrieved with the same ordering followed while forming a cluster:

$$R(y', p_i) = \min(d(y', p_i)). \quad (10)$$

If the reachability distance $R(y', p_i)$ is lesser than the threshold value T , then the point is considered inside the cluster, and if it is higher than the threshold value, it is considered

as an anomaly:

$$Z = \begin{cases} 0 & \text{if } R(y', p_i) > T \\ 1 & \text{if } R(y', p_i) < T \end{cases} \quad (11)$$

Our work implements real-time data processing using OPTICS for anomaly detection with a dedicated OPTICS instance for monitoring each device as shown in Fig. 4. We consider the identification of following soft failure types in this work: (i) *gain degradation* in the amplifiers (pre- and booster amplifiers), where the amplifier gain is lower than the expected value; (ii) *increase in attenuation* in the WSS; (iii) *signal attenuation* in the booster-amplifier; and (iv) *launch power degradation* in the optical terminals, where the optical transmission power is lower than the expected value. The consequence of soft failure is that the provisioned optical channel still uses the link but with underutilization of the resources. With continuous performance data monitoring, the deviation in values from their normal state is identified using OPTICS. At regular intervals, each OPTICS instance monitoring each device outputs a standard one-hot encoded vector that represents the status of the PM metrics. The standard vector is the collection of various KPIs of the optical devices present in the network, where each device represents the status of the PM metrics corresponding to it. This proposed method helps to generalize the failure identification process during network expansion and service reconfigurations.

C. Failure Localization Using a GNN

We address the soft failure localization problem through a two-step process: (i) failure identification using unsupervised ML as discussed in Section 4.B.1 and (ii) use of a GNN to classify the failure type. Existing works have explored rule-based and classical neural network approaches for failure localization, but with the limitation that they operate only on predefined network topologies [17,18]. We propose two enhanced variants of a GNN [29] to identify the relationship between nodes that exhibit anomalies and the root cause of the anomaly.

1. Graph Neural Network—Graph Attention Network

GNNs are known for discovering the relationships among nodes in the network by modeling the network with nodes and edges. The GNN is extended with a graph attention (GAT) mechanism, allowing each node in the network to aggregate data from its neighboring nodes. This helps capture the relationships among various types of connected neighboring devices, including transponders, amplifiers, PSMs, and WSSs.

The GNN-GAT model is formulated as $G = \{N, A\}$, where N represents the optical nodes, and A represents the adjacency matrix encoding the connections in the physical network. Each node $i \in N$ is characterized by feature data h_i of size $f \in \mathbb{R}$, which is the result of the anomaly identification process using OPTICS. Equation (12) represents the current feature data of all nodes:

$$D_t = [h_1, h_2, \dots, h_N]. \quad (12)$$

In the GNN-GAT model, the initial step is the attention mechanism, where we calculate the attention score α_{ij} for each

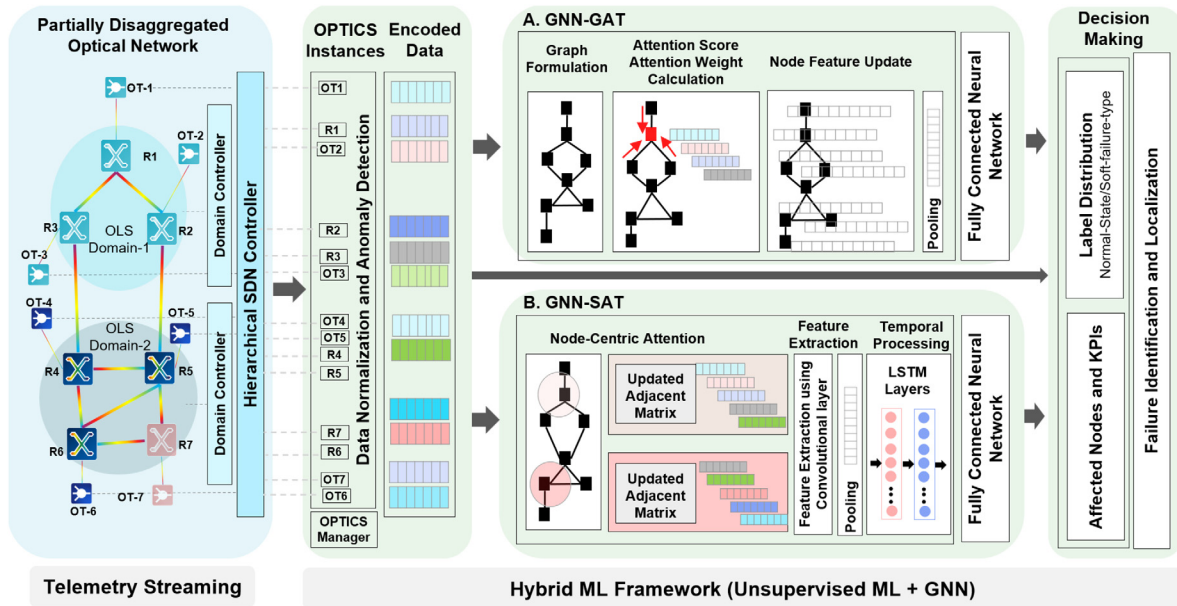


Fig. 4. Overall architecture—ONF-TAPI-based streaming and soft failure localization using hybrid ML framework in the optical transport network.

node i relative to its one-hop local neighborhood $j \in N$. The data of a node i and its neighbor j are concatenated as

$$z_{ij} = [h_i \parallel h_j] \in \mathbb{R}^{2f}. \quad (13)$$

The concatenated value z_{ij} is dot-multiplied with a learnable weight matrix W_a and is projected into a hidden space. A non-linear activation function σ [30] is applied over the dot product, enabling the learning of more complex relationships. During training, the model learns the interaction between the features of nodes i and j . A learnable attention vector $a^\top$ assigns a score to the interaction between nodes i and j through a dot product operation. This reduces the computed hidden value to a scalar, simplifying interpretation. The resulting scalar value α_{ij} is the attention score, defined as

$$\alpha_{ij} = a^\top \sigma(W_a \cdot z_{ij}). \quad (14)$$

To prevent domination by nodes with higher magnitudes, attention scores are normalized using the softmax function [31], as shown in Eq. (15). In this way, the features of all nodes in the graph are updated with the connected neighbors. Nodes with low attention scores contribute less to the feature aggregation, allowing the model to focus on the most relevant nodes:

$$\beta_{ij} = \frac{\exp(\alpha_{ij})}{\sum_{k \in N(i)} \exp(\alpha_{ik})}. \quad (15)$$

The state of each node is updated using the aggregated data from its neighbors h_j , weighted by the computed attention scores β_{ij} and transformed by the learnable weight matrix W_b . An activation function σ is applied to introduce non-linearity into the model during aggregation, as shown in the following equation:

$$h'_i = \sigma \left(\sum_{j \in N(i)} \beta_{ij} \cdot W_b h_j \right). \quad (16)$$

After the attention mechanism, each node in the graph has an updated feature vector h'_i . Max-pooling [32] is applied to reduce the dimensions, abstracting the graph representation into a single vector. This vector is then fed into fully connected neural network (FCNN) layers to transform the feature vector and obtain the label distribution probabilities for software failures, as shown in Fig. 4. For training the model, we have collected the data samples with respective soft failure types from the testbed [14]. During training, the cross-entropy loss between the model's predicted output and the ground truth is calculated. Gradients are computed from the cross-entropy loss, guiding the optimizer in updating learnable parameters to improve predictions.

The GNN-GAT model relies on a fixed adjacency matrix to perform the attention mechanism, which makes it less flexible to changes in the network, such as the addition or deletion of nodes or links. This also affects the implementation of a recurrent neural network (RNN) in GNN-GAT, as the addition or deletion of nodes and links affects the temporal pattern of the trained ML model. In addition, retraining of the entire supervised model for the modified network structure is computationally expensive and is not suitable for dynamic real-time monitoring.

2. Graph Neural Network—Sample, Aggregate, and Temporal

We propose a graph neural network—sample, aggregate, and temporal (GNN-SAT) as an alternative to GNN-GAT, as shown in Fig. 4, to incorporate the flexibility to apply the adjacency matrix in real time and enable the tracking of behavioral patterns of the nodes preceding failure for dynamic

network configurations. *Sample*—a region-based data selection strategy is introduced that incorporates a node-centric attention mechanism that prioritizes processing nodes exhibiting KPI deviations. A graph $G = \{N, A\}$ is formulated based on the network topology, where N represents the number of nodes and A denotes the adjacency matrix. Each node $i \in N$ has an input feature b_i of size $f \in N$, which is a one-hot encoded vector resulting from the failure identification assessment in Section 4.B.1. In the node-centric attention mechanism in GNN-SAT, each node in the network is iterated, and respective input feature b_i is processed. From the data, the GNN-SAT model identifies the nodes that exhibit anomalous behavior. The affected nodes along with their neighboring nodes are aggregated and form a standard matrix of size $m \times f$. *Aggregating* the neighbors helps capture the relationship between the nodes that exhibit anomalies and the corresponding neighbors during soft failure. This attention mechanism reduces node inputs of the entire network into a standard matrix that focuses on the region where soft failure has occurred. This was facilitated by the fault identification step in Section 4.B.1 where deviation in the KPI is captured.

During training, node inputs are processed in batches with batch size s and sequence length l to enable learning of the spatial and temporal pattern of nodes preceding SF. The processed batch of input data is represented as $E^{ts \times m \times f}$ passes through the convolution layer with learnable weights W_1 , and bias b_1 as represented

$$H_1 = \sigma.(W_1 * E^{ts \times m \times f} + b_1). \quad (17)$$

In order to extract the low-level spatial relationship between the affected nodes and their neighbors, another convolution layer is used with learnable weights W_2 and bias b_2 as shown in Eq. (18). Each convolution operation is followed by a dot product with the activation function σ as shown in Eqs. (17) and (18):

$$H_2 = \sigma.(W_2 * H_1 + b_2). \quad (18)$$

Max-pooling [32] is applied on H_2 , which effectively reduces the data to a vector Z of dimension $d \in R$ retaining the important features as well. As mentioned earlier, the node attention mechanism is repeated for sequence length $l \in R$, and the outcome from each iteration is stored in an array as given in Eq. (19):

$$g_t = [Z_1, Z_2, \dots, Z_l]. \quad (19)$$

To capture temporal relationships among the KPIs of the node preceding soft failure in real time, LSTM [33] is used in our proposed GNN-SAT model. LSTM, a type of recurrent neural network, incorporates a memory cell controlled by three gates: an input gate, a forget gate, and an output gate. At each time step, the LSTM processes three inputs: the current input g_t , the hidden state from the previous time step h'_{t-1} , and the cell state from the previous time step C_{t-1} . Forget gate: The first step in the LSTM is to determine what information should be discarded from the cell state. This decision is made by the forget gate, which computes a sigmoid function using the current input g_t and the previous hidden state h'_{t-1} , as shown

in Eq. (20). This is especially important during transitions between normal and failure states, and vice versa.

$$f_t = \sigma(W_f \cdot [h'_{t-1}, g_t] + b_f). \quad (20)$$

Input gate: The second step involves deciding what new information should be stored in the cell state. This is accomplished through the input gate, which consists of two layers: (i) a sigmoid layer and (ii) a tanh layer. These layers identify which values should be updated in the cell state, computed using g_t and h'_{t-1} as shown in Eq. (21) and Eq. (22):

$$i_t = \sigma(W_i \cdot [h'_{t-1}, g_t] + b_i), \quad (21)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h'_{t-1}, g_t] + b_C). \quad (22)$$

This is followed by the actual writing of data to the cell state C_t by combining Eqs. (20)–(22). This mechanism helps store the new context associated with a transitioned normal or failure state as shown in Eq. (23):

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t. \quad (23)$$

Output gate: The final step determines which part of the cell state will be output by the LSTM. The output gate consists of (i) a sigmoid layer in Eq. (24) and (ii) a tanh layer in Eq. (25), ensuring that the relevant portions of the transition state are carried forward to the next time step:

$$o_t = \sigma(W_o \cdot [h'_{t-1}, g_t] + b_o), \quad (24)$$

$$h'_t = o_t * \tanh(C_t). \quad (25)$$

The resulting output is passed through an FCNN to obtain the label distribution probability for soft-failure classification as shown in Fig. 4. The output from unsupervised ML and GNN label distribution is processed in the decision-making module to localize the SF precisely in the network. By integrating the LSTM module, GNN-SAT benefits from the ability to detect feature deviations and capture the temporal behavior of soft failure. The model is built with configurable parameters, including sequence length, batch size, and the input, hidden, and output dimensions of LSTM and FCNN layers. Further, the optimization is performed using Adam optimizer [34] to improve training and ensure faster convergence.

Algorithm 1 depicts the workflow of TAPI-based telemetry streaming followed by the hybrid ML framework in a partially disaggregated optical network. The initial step is the instantiation of the developed telemetry agent in the CP for telemetry streaming using gNMI/gRPC complied with TAPI-based data models. The performance metrics interchangeably referred to as KPIs in the paper are shown in Table 4, are streamed using a subscription mechanism from optical devices of different domains every 10 s, and are stored in the time-series DB in the MP. An OPTICS manager application is developed following a microservice architecture to manage multiple OPTICS container instances. It is integrated with the hierarchical SDN controller and exposes APIs to receive the IP address and number of training samples n required to instantiate OPTICS ML instances for monitoring individual devices. Additional

Algorithm 1. SF Localization Using the Hybrid ML Framework

```

1: Initialize TAPI-based telemetry streaming agent(s)
2:  $resp \leftarrow$  REST request to OPTICS manager with  $ip$  list, and  $n$  sample
3: Instantiation of OPTICS ML instances for device monitoring
4: procedure FAILUREIDENTIFICATION( $ip, n$ )
5:    $X \leftarrow$  checks for  $n$  updated training data
6:    $X', scaler \leftarrow$  Apply min-max scaler to normalize
7:    $T \leftarrow$  Train data based on OPTICS
8:   while True do
9:      $Y \leftarrow$  Retrieve real-time data
10:     $Y' \leftarrow$  Normalize the data using  $scaler$ 
11:     $Z \leftarrow$  Test whether  $Y'$  is in the cluster
12:     $h \leftarrow$  Find deviated KPIs, encode and update DB
13:    sleep for sample interval  $sleep$ 
14: procedure SFTypeIDENTIFICATION( $N, A$ )
15:   if Approach is GNN-GAT then
16:     Formulate a graph  $G=[N,A]$  based on the network
17:      $D_t \leftarrow$  Retrieve data of all nodes from time-series DB
18:     for all node  $i$  in  $N$  do
19:       for all node  $j$  in  $N$  do  $\triangleright$  if  $i$  and  $j$  are neighbors
20:          $\alpha_{ij} \leftarrow$  Node attention score calculation
21:        $\beta_{ij} \leftarrow$  Attention score normalization
22:       for all node  $j$  in  $N$  do
23:         for all node  $j$  in  $N$  do  $\triangleright$  if  $i$  and  $j$  are neighbors
24:            $h'_i \leftarrow$  Update node state with data aggregation
25:          $q' \leftarrow$  Down-sampling using MaxPooling
26:          $O_{GNN-GAT} \leftarrow$  SF label distribution using FCNN
27:       return  $O_{GNN-GAT}$ 
28:   else if Approach is GNN-SAT then
29:     Formulate a graph  $G=[N,A]$  based on the network
30:      $E^{l \times s \times N \times f} \leftarrow$  Retrieve data from time-series DB
31:     for all sequence  $seq$  in  $l$  do
32:       for all batch  $batch$  in  $s$  do
33:         for all node  $i$  in  $N$  do
34:            $a\_nodes \leftarrow$  Nodes with anomaly
35:           for all node  $i$  in  $a\_nodes$  do
36:              $S^{m \times f} \leftarrow$  Formulate std. matrix
37:              $E'^{s \times m \times f} \leftarrow$  Append reduced matrix  $S^{m \times f}$ 
38:              $H_1, H_2 \leftarrow$  Spatial feature extraction
39:              $g_t \leftarrow$  Down-sampling using MaxPooling
40:              $h'_t \leftarrow$  Temporal pattern extraction and classification
41:              $O_{GNN-SAT} \leftarrow$  SF label distribution using FCNN
42:           return  $O_{GNN-SAT}$ 
43:   output  $\leftarrow$  SF is localized by identifying the deviated KPIs from OPTICS and type of SF from GNN.

```

APIs are implemented to reset and delete OPTICS ML containers when required. The training data X for each OPTICS instance depend on the current device configuration, which can be updated locally or retrieved from the time-series DB (line 5). The data X are normalized to X' using a min-max scaler as shown in Eq. (2) and are used to train the OPTICS model to calculate the threshold reachability distance T using Eqs. (3)–(8) (lines 6–7).

The real-time PM data of the device Y are retrieved from the DB every 10 s and normalized with the same scaler $scaler$ to produce Y' . Calculating Z helps to differentiate anomaly data from normal data using Eqs. (10) and (11). Data Y' are

encoded as h and updated to the DB for further processing. In the event of an anomaly, the deviating KPIs are identified and updated to the DB. This is followed by the identification of the SF type with either of the proposed GNN approaches, (i) GNN-GAT or (ii) GNN-SAT.

The GNN-GAT formulates the graph $G = \{N, A\}$ and retrieves the encoded data of all nodes D_t from the time-series DB (line 17). The attention score α_{ij} is calculated and normalized to a value β_{ij} for all nodes N in the network using Eqs. (13)–(15). The data of all nodes are updated again using data aggregation (line 24) and down-sampled using the MaxPooling technique resulting in the hidden state q' . The hidden state data q' are applied to FCNN for the calculation of probability of SF classes $O_{GNN-GAT}$. The alternative approach proposed for SF type identification is the GNN-SAT, where the $l + s$ samples of all N nodes are retrieved from the DB to process the data for the sequence length l and the batch size s . The node-centric attention mechanism is performed on each sample to identify the anomaly nodes a_nodes and formulate a standard matrix $S^{m \times f}$ including the data of the neighboring nodes. While processing the anomaly nodes and identifying the data of the respective neighboring nodes, the current adjacency matrix A is used, which helps to apply GNN-SAT in dynamic network configurations. The operation is performed for batch size s where dimensionally reduced input data $E'^{s \times m \times f}$ are obtained. Two layers of convolution operation are performed on the data $E'^{s \times m \times f}$ to extract the spatial characteristics of the data resulting in H_1 and H_2 . This is followed by MaxPooling to effectively reduce the dimension of the data for further processing. The entire process is repeated for sequence length l resulting in g_t , which is passed to the LSTM module to produce the hidden state h'_t . The SF probability distribution $O_{GNN-SAT}$ is calculated by passing the hidden state data h'_t into a FCNN. During the training phase of both GNN models, the cross-entropy loss value is calculated based on the ground-truth SF labels, and the optimizer is used to fine-tune the learnable parameters in the model.

5. EVALUATION

The experiment is carried out on a partially disaggregated optical transport network located at the Fraunhofer research institute testbed, Berlin [14]. The DP consists of two OLS domains with each domain having three ROADM nodes connected in a ring fashion and is controlled by dedicated OLS domain controller instances. An optical terminal is connected to each ROADM node in the OLS, and the overall architecture of the network is shown in Fig. 4. The TeraFlowSDN controller [27] is utilized as a hierarchical SDN controller incorporating advancements in the TAPI interface for network configuration and telemetry streaming. The developed GNN models are trained on a server with an Intel Xeon processor, 40 cores, and 128 GB RAM. The telemetry streaming agent, OLS domain controllers, hierarchical SDN controller, and ML applications are deployed on individual Linux machines, each equipped with 8 GB RAM and a 4-core CPU.

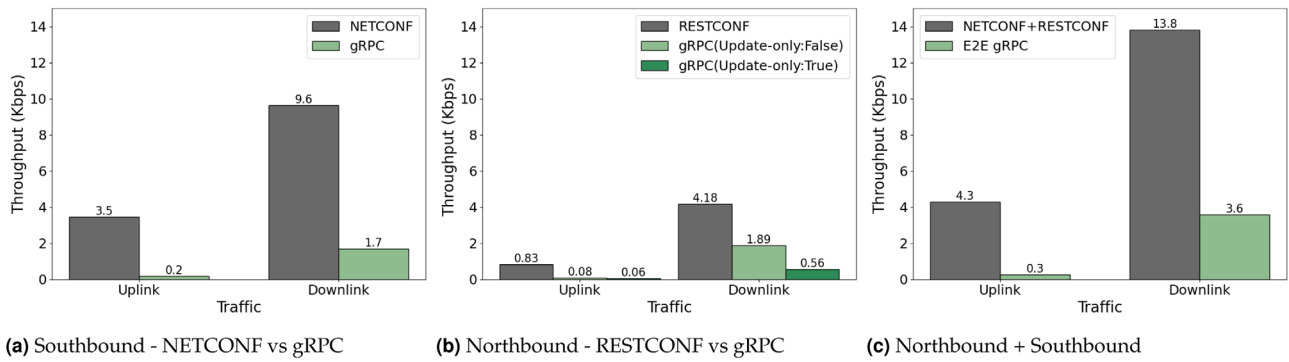


Fig. 5. End-to-end gNMI/gRPC-based telemetry streaming.

A. Telemetry Streaming

In the southbound between the DP and the CP, the throughput is evaluated between NETCONF and gNMI/gRPC-based telemetry data exchange. The gNMI/gRPC capability is configured in all DP components in the testbed to support telemetry streaming encrypted over TLSv1.3. The agent application developed is installed alongside the OLS controller to support the retrieval of data from devices using gNMI/gRPC and stream it to the MP complying with the TAPI specifications [4]. To ensure a fair comparison between NETCONF and gRPC, NETCONF requests are accompanied by an XML filter to retrieve only the PM data from the OTs listed in Table 4. Similarly, the gRPC subscription request is defined with XPath to retrieve only the intended PM data. The throughput of both protocols for retrieving PM data from the three OT devices in OLS domain-1 (OT-1, OT-2, and OT-3) is observed for a total of 10 min, with a sampling interval of 10 s. Throughout this work, we refer to a client-to-server request as an uplink and a server-to-client response as a downlink. As shown in Fig. 5(a), NETCONF consumes about 3.5 kbps and 9.6 kbps of traffic in the uplink and downlink transactions, respectively, whereas gRPC consumes 0.2 kbps for the uplink and 1.7 kbps for the downlink reducing the traffic load by a factor of 94.2% and 82.29% in uplink and downlink.

In the northbound between the MP and the CP, two challenges are addressed: (a) supporting the TAPI data model to increase DP visibility to the MP and (b) replacing RESTCONF-based PM data retrieval with gNMI/gRPC. The client application in the MP continuously listens for PM data from the device using a unified TAPI-gNMI-streaming data model [4] from the CP agent via gRPC. For ideal comparison, a REST-based telemetry data exchange application using the same data model is developed for benchmarking purposes. As speculated in ONF-TAPI [4], more telemetry traffic can be conserved if the client receives PM data only when there is an update in the value. This is implemented with a control variable `update_only: boolean` as described in the TAPI specifications [4] where the value *False* means that the client receives PM data every 10 s, and *True* means that the client only receives data when there is an update. In Fig. 5(b), the throughput consumption for the data streamed from three OT devices in DP is also streamed to MP for 10 min with

RESTCONF and gRPC is plotted. The PM data traffic in the northbound is decreased when compared to the southbound as the agent ignores payload overhead from the southbound. It is observed that gRPC(False) and gRPC(True) reduce the traffic load by a factor of 60.6% times and 87.6% when compared to RESTCONF.

The end-to-end telemetry data throughput between text-based protocols (such as NETCONF and RESTCONF) and binary encoded protocol-based methods (such as gRPC) are compared in Fig. 5(c). The evaluation shows that the end-to-end gNMI/gRPC-based streaming reduces the traffic load by a factor of 78.4% when compared to using NETCONF and RESTCONF in southbound and northbound, respectively. The novelty of gNMI/gRPC-based streaming is more apparent in a larger network. For example, monitoring 100 OTs with a sampling interval of 10 s generates about 2.36 and 0.5 TB of telemetry data for 1 year if we use NETCONF/RESTCONF and gNMI/gRPC, respectively. The introduction of gNMI/gRPC conserves about 1.87 TB of telemetry traffic in a year for monitoring OTs alone and showcases the efficiency of gNMI/gRPC over the text-based protocols in telemetry streaming.

B. Failure Identification Using OPTICS

The proposed unsupervised ML model is evaluated based on several key aspects: (a) multivariate data, (b) changes in the number of features due to reconfigurations, (c) efficient model training, (d) scalability, and (e) flexibility to monitor different devices.

To support these evaluations, the performance metrics of OT and OLS components mentioned in Table 4 are retrieved using the telemetry agent as described in Section 4.A and stored in a time-series DB in TFS every 10 s. Through OPTICS manager, an OPTICS ML instance container is deployed to monitor each optical device in the DP, as shown in Fig. 4. The implemented OPTICS algorithm is not limited to a number of metrics to be monitored, as this varies based on (a) the type of optical device and (b) the number of physical and logical interfaces configured in the device. After the container is instantiated, the model checks for updated training data, either locally or from the time-series DB. After successful training, the live data from the DB is retrieved periodically every 10 s and tested for anomaly.

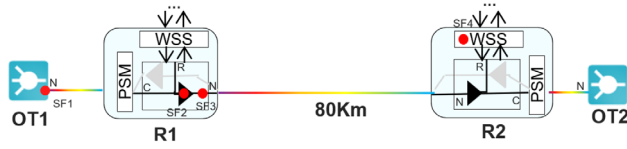


Fig. 6. Modeling of soft failure for behavioral analysis.

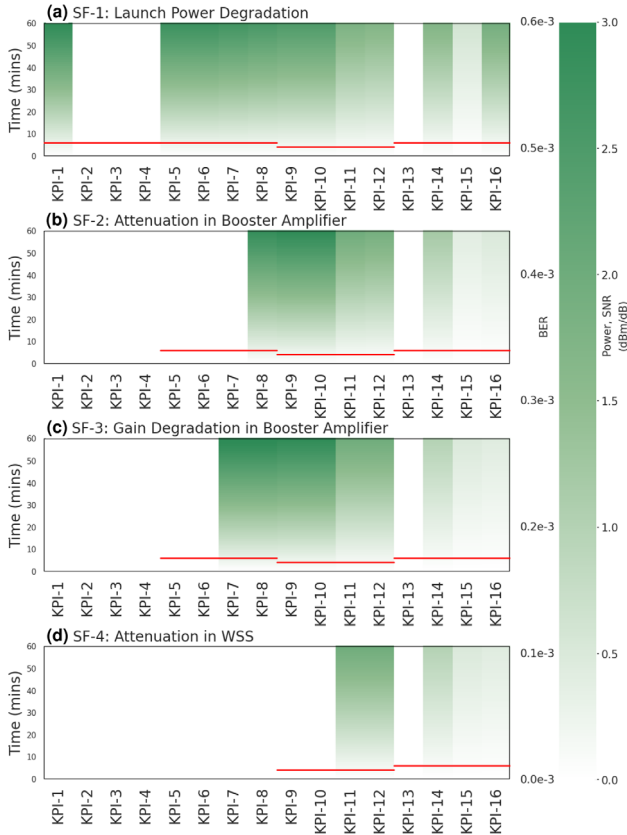


Fig. 7. Time-series KPI deviations for modeled SF scenarios.

A series of SF scenarios were modeled and introduced in the optical network segment between OT1 and OT2. This segment spans ROADM R1, an 80 km fiber spool, and ROADM R2 as shown in Fig. 6. The soft failures include SF1: launch power degradation, SF2: increased attenuation in the booster amplifier, SF3: gain degradation in the amplifier, and SF4: increased attenuation in the WSS. Soft failures are simulated by gradually degrading device performance to observe the resulting network behavior. The impact of these failures on KPIs across the nodes is monitored and important KPIs are shown in Fig. 7. The KPIs (KPI-1 to KPI-4 and KPI-13 to KPI-16) correspond to OT1 and OT2, representing key network port metrics: optical power TX, optical power RX, SNR, and BER. KPIs KPI-5 to KPI-8 are associated with R1 and denote amplifier-related measurements, including client port input power, booster amplifier input power, booster amplifier output power, and network port output power. Meanwhile, KPIs KPI-9 to KPI-12 correspond to R2, capturing the amplifier network port input power, pre-amplifier mid-stage output power, WSS output power, and amplifier client port output power.

In the SF1 scenario, the transmission power at OT1 is linearly reduced from 5 to 2 dBm over 60 min, with degradation every 30 s. This results in observable deviations in downstream components (ROADM1, ROADM2, and OT2), detected by individual OPTICS instances monitoring each device. A device is flagged as anomalous only if the KPI deviation persists over a sustained period, which is configurable to reflect the gradual nature of soft failures in a real network [35]. Similarly, SF2, SF3, and SF4 cause deviations in the KPIs of their respective downstream components. Deviations in input/output power or SNR of approximately 0.3 dBm/dB are reliably detected by the OPTICS instances, as shown in Fig. 7. Each OPTICS instance samples data every 30 s, encodes KPI deviations and device status into a one-hot vector, and stores it in a time-series DB for further processing using a GNN.

To evaluate the influence of training data size and threshold selection in fault identification, a dataset of 1000 samples was used. This dataset includes known KPI deviations caused by launch power degradation at the OT (SF1) and the effect of amplifier gain degradation at the OT (SF3). Approximately 10% of the data reflects anomalous behavior. The OPTICS algorithm was tested with varying training sample sizes (300, 600, 900, 1200, and 1500) collected from the OT. The true positive rate (TPR), defined as the number of correctly identified anomalies divided by the total number of actual anomalies, was calculated. As shown in Fig. 8(a), the TPR stabilizes for training sizes above 600, indicating the model's robustness with limited training data. However, increasing the threshold value T leads to a decrease in TPR, highlighting the importance of selecting an appropriate threshold. A threshold reachability distance of $T = 0.4$ was determined using 600 training samples, as described in Eq. (6). To further understand this behavior, reachability distances for both training and live data were visualized using the dendrogram plot in Fig. 8(b). In the case of SF1, only the optical transmission power deviates, resulting in smaller reachability distances. In contrast, SF3 affects multiple KPIs (SNR, receive power, BER) simultaneously, leading to significantly larger deviations. Since KPI values are projected into a multi-dimensional space in OPTICS, the reachability distance is naturally higher for SF3 compared to SF1. During the early stages of SF1, the deviation is minimal and may not be flagged as anomalous, which contributes to a lower TPR.

The scalability of Docker containers running an OPTICS instance to monitor the OT/OLS device is evaluated by measuring the average CPU and memory utilization of the containers. For benchmarking purposes, each device is monitored by more than one OPTICS container instance to showcase the compute resource utilization. A single container uses only about 0.1% of the CPU, and 25 containers monitoring 25 OT/OLS devices utilize approximately 3.74% as seen in Fig. 8(c). Similarly, memory utilization is observed, where 2.14 GB of main memory as shown in Fig. 8(d) is used by 25 container deployments monitoring 25 OLS/OT devices. This scalability and flexibility make the framework well suitable for real-time monitoring of larger networks and further assist in localizing soft failures using a GNN.

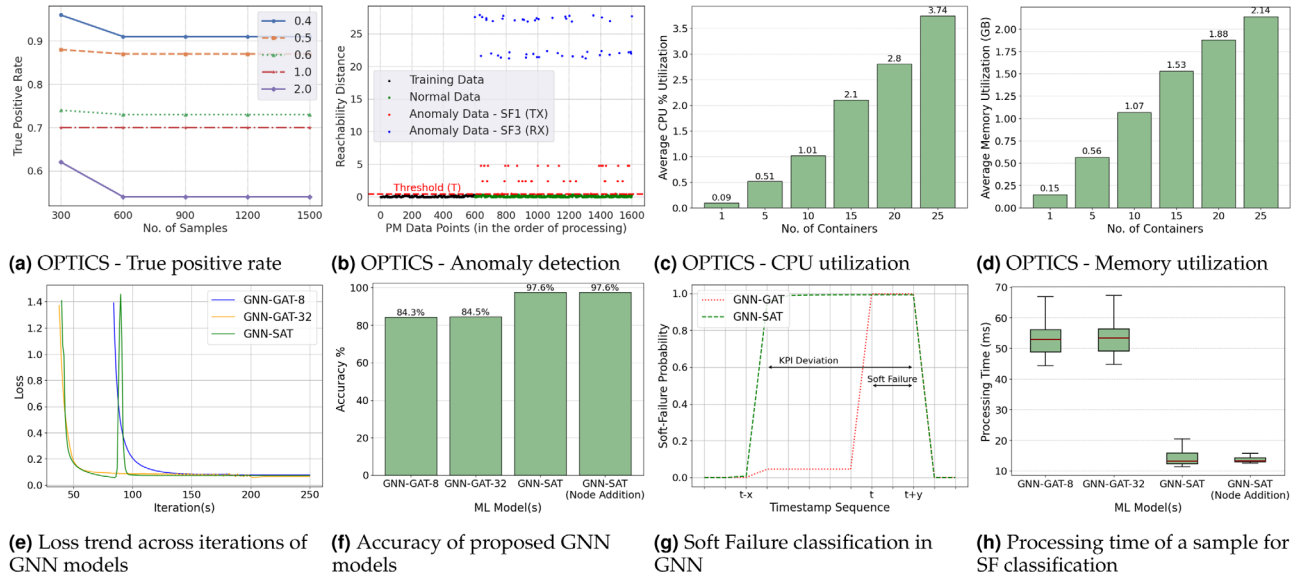


Fig. 8. Hybrid ML framework with clustering-based failure identification and GNN-based soft-failure classification.

C. Failure Localization Using a GNN

In the testbed, four soft failure scenarios are introduced as mentioned in Section 4.B.1 to observe their effects on the nodes and identify the pattern of the affected parameters in different components. This helps in understanding the deviated KPIs under different SF scenarios. Although anomaly detection in the previous section identifies affected nodes and parameter deviations, determining the root cause requires analyzing the relationships between them. This is investigated for the following GNN variants: the developed GNN-GAT model as described in Section 4.C.1 with the hidden state dimensions of 8 and 32 (denoted as GNN-GAT-8 and GNN-GAT-32), and GNN-SAT is observed for the standard network and with a two virtual node addition denoted as [GNN-SAT and GNN-SAT(node-addition)].

The testbed was modeled as a graph with nodes (6 OTs, 6 ROADMs) and edges from the network topology as shown in Fig. 4. The encoded output data h from unsupervised ML indicating the state of each device are given as an input to nodes in the GNN. A graph attention mechanism is applied as described in Section 4.C.1. Each node feature is updated with attention scores that are calculated based on the neighboring nodes, producing a hidden state h'_i . This hidden state is fed into FCNN, which consists of three fully connected layers, including two hidden layers with 16 neurons each. The output $O_{\text{GNN-GAT}}$ from FCNN of dimension 5 gives the distribution probability of the target class among one normal class and four SF classes. Unlike GNN-GAT, GNN-SAT uses a node-centric attention mechanism, eliminating the need to process the entire graph. The input to GNN-SAT is the result of the OPTICS model stored in time-series DB. The input data $E^{l \times s \times N \times f}$ to GNN-SAT are defined as the data with a sequence length $l = 5$ and batch size $s = 32$, number of nodes $N = 12$, and encoded KPI data dimension $f = 8$. The N value can be updated during the addition or deletion of nodes. GNN-SAT processes the affected region of the network by processing nodes that

exhibit KPI deviations and forms a standard matrix S^{sf} with the features of the affected nodes. The process is repeated for a batch size of s and the result from each iteration is aggregated. To learn spatial features, the aggregated data $E^{l \times s \times m \times f}$ with ($s = 32$, $m = 8$, $f = 8$) are passed through two convolution layers with 16 and 32 filters and followed by down-sampling of the data to a hidden state g_t . The data are iterated for the past l timestamp sequence to learn the temporal characteristics of the data. The temporal property is enabled by using two LSTM layers (input_size = 512, hidden_size = 64) to process hidden state of nodes and capture the behavior of the data preceding SF. The output h'_t of the LSTM is passed through a single-layer FCNN (input_size = 64, output_size = 5) for target-class or SF-type probability distribution $O_{\text{GNN-SAT}}$. The identified SF-type from GNN, along with anomaly nodes and KPI deviations detected by the unsupervised ML model, is used in the decision-making process to localize SF. During training of the model, both the GNN-GAT and GNN-SAT algorithms are trained for 1000 epochs, and the Adam [34] optimizer with a learning rate of 0.001 is used to optimize the learnable parameters in the attention mechanism, convolution, FCNN, and LSTM operations.

The behavior of devices under the four modeled SF conditions is analyzed through time-series deviations in KPIs as described in Section 5.B. After each sampling interval, the OPTICS instance generates an 8-bit one-hot encoded vector, where 5 bits represent the KPI state and 3 bits represent the device state. These encoded data are periodically fed into the GNN as node input that models the actual network topology. To evaluate the GNN models, soft failures are simulated at different locations within the network. A dataset comprising 1500 time-series samples is generated with SF scenarios occurring in different parts of the network, with 500 samples used for training and 1000 for testing, with ground-truth labels available at every timestamp. The convergence of models is analyzed by plotting the loss against each iteration during the training process. GNN-GAT with hidden state dimensions of

8 and 32 converges to 0.07 and 0.06 after 200 iterations, where GNN-SAT converges to 0.07, as shown in Fig. 8(e). GAT-32 exhibits faster convergence than GAT-8 and GNN-SAT due to its hidden state with a larger dimension. The accuracy of the models in predicting the target class is evaluated using time-series data of about 1000 samples with different SF scenarios. As shown in Fig. 8(f), GAT-8 and GAT-32 achieve an accuracy of approximately 84.3% and 84.5%, where the GNN-SAT and GNN-SAT with node addition achieve an accuracy of 97.6%. Despite the slower convergence of GNN-SAT, it achieves a higher accuracy than GNN-GAT due to its ability to model temporal dependencies through LSTM, allowing it to identify soft failures earlier. For GNN-SAT (node-addition), a virtual ROADM (R7) and optical terminal (OT-7) are added with configurations similar to existing nodes (OT-6 and R6) as shown in Fig. 4. A custom script periodically updates their PM data to the time-series DB in a required format. The OPTICS instances are instantiated to monitor the new nodes, and GNN-SAT accommodates the updated network topology instantaneously by adjusting the adjacency matrix. Due to SAT properties of the proposed GNN model, retraining the model is not required despite network modification. The lower accuracy of GNN-GAT compared to GNN-SAT highlights the importance of RNN to identify KPI deviations preceding SF.

The probability distribution over the target classes produced by the GNN models is monitored at each timestamp. By learning the temporal behavior of the SF, the GNN-SAT identifies failures during KPI deviations from $t - x$ effectively anticipating the SF before it is fully apparent at time t , as shown in Fig. 8(g). The time offset x varies on the type of SF and cannot be predefined, as SF occurrence is influenced by multiple device-specific parameters. GNN-GAT also detects a low probability of SF based on KPI deviations learned through the attention mechanism but confirms SF only during its occurrence between time t and $t + y$ as shown in Fig. 8(g). The processing time for evaluating the samples is critical, as this operation is performed periodically in real time. A dataset with 1000 samples is tested, and it is observed that the average processing time for GAT-8, and GAT-32 is approximately 56.41 ms and 58.02 ms, respectively. This delay is due to entire graph-level operations, including GAT and aggregation. However, GNN-SAT benefits from its node-centric attention mechanism and achieves a significantly shorter processing time of approximately 15.28 ms, as shown in Fig. 8(h), making it more efficient.

GNN-GAT is less flexible for dynamic topology as it requires retraining of the model when nodes are added or removed from the network. Additionally, integrating LSTM into GNN-GAT introduces complexity as changes in topology affect hidden state dimensions, leading to uncertainties in the SF classification. In contrast, GNN-SAT eliminates the need to process the entire graph by using SAT techniques that focus on the affected region of the network. This simplifies the integration of LSTM, enabling proactive SF identification.

6. CONCLUSION

We presented a unified ONF-TAPI compliant telemetry streaming in a partially disaggregated optical transport

network, leveraging a hybrid ML framework for SF localization. The end-to-end gNMI/gRPC-based telemetry streaming is evaluated, showing a 78.4% reduction in traffic load compared to traditional NETCONF and RESTCONF approaches. This ONF-TAPI-based streaming enhances the visibility of device KPIs at the management plane, enabling operators to monitor and control multiple network domains across different vendors. The proposed ML framework combines unsupervised learning for anomaly detection with an inductive GNN-SAT model for SF localization. With a minimal training data requirement, the distributed unsupervised ML approach enables continuous monitoring for anomaly detection, effectively supporting dynamic network conditions. The inductive nature of the GNN-SAT model ensures that accuracy remains consistent, even as new nodes are added to the network, demonstrating stability during network reconfiguration. The solution has the scope to identify and locate multiple failures simultaneously and presents a promising research direction for future work.

Funding. The European Smart Networks and Services Joint Undertaking (SNS JU) (101096120); Bundesministerium für Forschung, Technologie und Raumfahrt (16KIS2098).

Disclosures. The authors declare no conflicts of interest.

REFERENCES

1. H. Zaid, A. Moawad, B. Shariati, *et al.*, "Autonomous service provisioning and self-healing in multi-band multi-domain IPOWDM networks," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2025), paper Th1H.
2. V. Karunakaran, S. K. Patri, S. Zimmermann, *et al.*, "OpenROADM for disaggregated optical networks: challenges, requirements and evaluation," in *Photonic Networks: 24th ITG-Symposium* (2023).
3. OpenROADM Multi-Source Agreement, "OpenROADM: interoperable optical networks using standardized open models" (2023).
4. ONF, "Transport API (TAPI) SDK v2.5.1 release," Open Networking Foundation (2024).
5. A. D'Amico, S. Straullu, G. Borracchini, *et al.*, "Enhancing lightpath QoT computation with machine learning in partially disaggregated optical networks," *IEEE Open J. Commun. Soc.* **2**, 564–574 (2021).
6. R. Casellas, R. Martínez, R. Vilalta, *et al.*, "Advances in SDN control and telemetry for beyond 100G disaggregated optical networks invited," *J. Opt. Commun. Netw.* **14**, C23–C37 (2022).
7. L. Velasco, P. Gonzalez, and M. Ruiz, "Distributed intelligence for pervasive optical network telemetry," *J. Opt. Commun. Netw.* **15**, 676–686 (2023).
8. P. Lechowicz, C. Natalino, V. Karunakaran, *et al.*, "Trade-offs in implementing unsupervised anomaly detection with TAPI-based streaming telemetry," in *IEEE 25th International Conference on High Performance Switching and Routing (HPSR)* (2024), pp. 13–18.
9. E. Etezadi, C. Natalino, V. Karunakaran, *et al.*, "Demonstration of DRL-based intelligent spectrum management over a T-API-enabled optical network digital twin," in *49th European Conference on Optical Communications (ECOC)* (2023), pp. 1730–1733.
10. R. Martínez, C. Hernández-Chulde, R. Casellas, *et al.*, "Enhancing network performance and reducing power consumption in elastic optical networks with deep reinforcement learning," in *24th International IFIP Conference on Optical Network Design and Modeling (ONDM)* (2024).
11. R. Casellas, R. Martínez, R. Vilalta, *et al.*, "Abstraction and control of multi-domain disaggregated optical networks with OpenROADM device models," *J. Lightwave Technol.* **38**, 2606–2615 (2020).
12. J. Kundrat, M. Vasko, R. Krejci, *et al.*, "Opening up ROADMs: streaming telemetry invited," *J. Opt. Commun. Netw.* **13**, E81–E93 (2021).

13. C. Manso, R. Munoz, N. Yoshikane, *et al.*, "TAPI-enabled SDN control for partially disaggregated multi-domain (OLS) and multi-layer (WDM over SDM) optical networks invited," *J. Opt. Commun. Netw.* **13**, A21–A33 (2021).
14. B. Shariati, H. Qarawlus, S. Biehs, *et al.*, "Telemetry framework with data sovereignty features," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2023), paper M3G.2.
15. Z. Li, Y. Zhao, Y. Li, *et al.*, "Fault localization based on knowledge graph in software-defined optical networks," *J. Lightwave Technol.* **39**, 4236–4246 (2021).
16. M. Ibrahim, F. Temiz, F. Musumeci, *et al.*, "Vertical federated learning for failure localization in partially disaggregated optical networks," in *IEEE 25th International Conference on High Performance Switching and Routing (HPSR)* (2024).
17. R. Wang, Q. Zhang, J. Zhang, *et al.*, "Multi-failure localization in high-degree ROADM-based optical networks using rules-informed neural networks," *IEEE J. Sel. Areas Commun.* **43**, 1738–1754 (2025).
18. S. Shen, J. Han, K. Bardhi, *et al.*, "Unified monitoring and telemetry platform supporting network intelligence in optical networks," *J. Opt. Commun. Netw.* **17**, 139–151 (2025).
19. P. Castoldi, F. Cugini, M. Gharbaoui, *et al.*, "Shaping the future of optical networks by integrating SDN, telemetry, and AI," *J. Opt. Commun. Netw.* **17**, C51–C61 (2025).
20. M. Balanici, P. Safari, B. Shariati, *et al.*, "Live demonstration of autonomous link-capacity adjustment in optical metro-aggregation networks," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2024), paper M3Z.3.
21. OpenConfig Working Group, "OpenConfig: vendor-neutral, model-driven network management" (2023).
22. M. G. Bhat, S. Bhattacharjee, C. Gündoğan, *et al.*, "CORECONF, NETCONF, and RESTCONF: benchmarking network orchestration in constrained IIoT devices," *IEEE Internet Things J.* **11**, 13082–13090 (2024).
23. S. Jackson, N. Cummings, and S. Khan, "Streaming technologies and serialization protocols: empirical performance analysis," *arXiv* (2024).
24. D. Rafique and L. Velasco, "Machine learning for network automation: overview, architecture, and applications [Invited Tutorial]," *J. Opt. Commun. Netw.* **10**, D126–D143 (2018).
25. "gRPC network management interface (gNMI)," Github (2021), <https://github.com/openconfig/gnmi>.
26. "NetEmu," Github (2024), <https://github.com/advaoptical/netemu>.
27. V. Karunakaran, C. Natalino, B. Shariati, *et al.*, "TAPI-based telemetry streaming in multi-domain optical transport network," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2024), paper M3Z.9.
28. F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: machine learning in Python" (2011).
29. X. Xu, H. Chen, J. E. Simsarian, *et al.*, "Optical network diagnostics using graph neural networks and natural language processing," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2023), paper M3G.5.
30. A. L. Maas, "Rectifier nonlinearities improve neural network acoustic models," in *Proceedings of the 30th International Conference on Machine Learning* (2013).
31. J. S. Bridle, "Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition," in *Neurocomputing: Algorithms, Architectures and Applications* (Springer, 1990), pp. 227–236.
32. Y. LeCun, B. Boser, J. Denker, *et al.*, "Handwritten digit recognition with a back-propagation network," in *Advances in Neural Information Processing Systems* (1989), Vol. **2**.
33. S. Hochreiter, "Long short-term memory," in *Neural Computation* (MIT, 1997).
34. D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," *arXiv* (2014).
35. S. Barzegar, M. Ruiz, and L. Velasco Esteban, "Soft-failure localization and time-dependent degradation detection for network diagnosis," in *International Conference on Transparent Optical Networks (ICTON)* (2020).